Video game emulation: Using FPGA technology with open source tools

Kerim Bavcic, Nedim Osmic, Harun Kovacevic
Department for Automatic Control and Electronics
Faculty of Electrical Engineering, University of Sarajevo
Sarajevo, Bosnia and Herzegovina
{*kbavcic1, nosmic, hkovacevic1*}*@etf.unsa.ba*

Anel Hadziaganovic
Embedded technology department
Everest Consulting doo
Sarajevo, Bosnia and Herzegovina
anel@everest.ba

2026 25th International Symposium INFOTEH-JAHORINA (INFOTEH) | 979-8-3315-6964-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/INFOTEH68759.2026.11477622

***Abstract*—This paper explores the application of FPGA (Field Programmable Gate Array) technology based on the Basys 3 board in video game development. Given that the hardware of most classic games is no longer functional, the programmable nature of FPGA allows for precise replication, allowing modern devices to run these games and provide an authentic gaming experience. The paper presents the design and emulation of a video game on the Basys 3 FPGA development system. It also examines the use of open-source tools for the development of FPGA applications, comparing them with commercial alternatives. The Basys 3 system was prepared for game emulation by implementing a RISC-V processor, a VGA controller, and memory in Verilog. The game was initially developed in C for Windows and later ported to the FPGA environment, with particular attention paid to memory management to ensure proper image display via the VGA controller. The results demonstrate that FPGA systems are viable platforms for video game emulation and complex application development, with open-source tools proving efficient and effective.**

***Index Terms*—Video game development; Basys 3 FPGA board; openXC7 toolchain; RISC-V SoC; VGA controller.**

I. Introduction

Embedded systems are independent computer hardware setups engineered to carry out a particular task in a repetitive manner. A typical system on a chip consists of a processor, memory and peripherals connected to a common bus and external communication interfaces. Field Programmable Gate Array - FPGA can be reprogrammed according to the desired application or functional requirements after production. Furthermore, FPGA enables the integration of soft-core processors [1]–[3], which is also the case in our work. FPGAs, by their very nature, allow operations to be executed in parallel [4], which gives them certain advantages over classic microcontrollers where processing takes place sequentially [1].

Why use FPGAs to develop games? First, emulation is at hardware level, which grants more accurate and efficient gaming experience comparing to software emulation. Besides that, FPGAs are mostly known for their flexibility and reprogrammability, which means they can be reconfigured in real-time allowing developers to specifically optimize hardware. This feature can improve performance and functionality. Also, parallel processing is also one of the most commonly know FPGA feature, therefore FPGAs are devices are very suitable for image and sound processing [5]–[10].

Comparing FPGAs with standard CPUs, which are unable to perform parallel processing, they can perform multiple operations simultaneously. This grants better performance and user experience especially in games that require high computational power. But one of the most important reasons for FPGA game development is the preservation of legacy games. It is known that most of historic games hardware is not usable, so FPGAs can accurately replicate it and allow classic games to be played on modern devices granting classic game experience.

Before emulating games on FPGA platforms, understanding the basics of game programming is crucial. The book [11] provides essential knowledge of game programming principles, industry patterns, and best practices, which are foundational for successful FPGA implementations. Also, in the MSC thesis [12], the main advantages of using FPGA technology for video game emulation are listed.

In paper [13] authors demonstrates the design and implementation of Tetris on an FPGA using VHDL. This study highlights the potential of FPGA for implementing logic games, providing insights into VHDL-based game programming on hardware platforms. In [14], a Ping-Pong game system is proposed, using an FPGA chip as the control core with minimal peripheral circuits. The simple yet flexible game design offers a practical and innovative example of FPGA's potential in entertainment. AI algorithms for game implementation on FPGA are explored in [15], specifically for the game "Blokus Duo." This research demonstrates the application of heuristic algorithms to FPGA platforms to meet design time constraints, offering valuable insights into AI implementation on hardware.

The development of simple game emulation systems on FPGA platforms is an excellent learning tool. The paper [16] explores emulating the game "Simon Says" on FPGA, illustrating its practical use for educational purposes. The acceleration of AI components in the game "Othello" is examined in [17], where the use of FPGA tablets is recommended for improving computation time in board games like "Go" and "Othello," highlighting the efficiency of FPGA over traditional processors in mobile devices.

This paper presents the design of a system based on FPGA technology that will be responsible for the emulation of the

The work is supported by $Chili.CHIPS^{ba}$ and Symbiotic EDA.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:59 UTC from IEEE Xplore. Restrictions apply.

TetriSaraj video game, which is inspired by the classic Tetris. It is a system made up of several components, such as RISC-V processor, flash memory, memory card, VGA controller. The components are described in detail below. The Basys 3 development system, which was used to develop the game, has 5 buttons in its structure that are used as inputs in interaction with the player, and the full-resolution monitor connected to the board represents the output on which the entire graphic interface of the game is displayed. In addition to this, an important aspect of the work is the comparison of open-source tools with commercial tools for the development of FPGA applications, as well as their influence and application in the development of more complex FPGA applications. The paper is organized as follows. The FPGA development tools are described in Section II. Section III explains video game emulation. In Section IV HW/SW interface is described. Emulation of video game based on FPGA is given in section V. The conclusions are written in section VI.

II. FPGA DEVELOPMENT TOOLS

A. Basys 3 Artix-7 FPGA Board

As Xilinx and Altera are the two main FPGA manufacturers in general, development board used in this work is Basys 3 from Digilent, shown on figure 1.

The board was build around Xilinx Artix-7 FPGA: XC7A35T-1CPG236C, which is known for its high performance and power efficiency. It has 4-digit 7-segment display, 5 user pushbuttons, 16 user LEDs, 16 user switches and a USB HID Host for mice, keyboards, and memory sticks. Besides that it has 4 Pmod ports, where 3 of them are standard 12-pin Pmod ports and the 4th one has dual purpose XADC signal or standard Pmod port. Basys 3 also has a 12-bit VGA output, USB-UART bridge, Serial Flash, USB-JTAG port used for FPGA programming and communication and an on-chip analog to digital converter (XADC). The basic specification of the board are: 33280 logic cells in 5200 slices (each slice contains four 6-input LUTs and 8 flip-flops), 1800 Kbits of fast block RAM, five clock management tiles (each with a phase-locked loop - PLL), 90 DSP slices and it's internal clock speed exceeds 450 MHz [18]. This allows developers to complete large number of designs without the need of additional hardware.

Digilent's Adept software was used for board programming, which is a powerful but easy to use tool. For design creation and modification Xilinx's Vivado Design Suite can be used, but the purpose of this paper, besides game emulation, was to use open source tools for development, in this case *openXC7 toolchain*.

B. openXC7 toolchain

It's an open source FPGA toolchain used for Xilinx 7 series FPGAs (Artix 7, Zynq7 and soon Spartan7) with added support for Kintex7 FPGAs (70T, 160T, 325T, 420T and 480T). OpenXC7 FPGA toolchain uses yosys and nextpnr which represent YosysHQ's products [19].

Yosys is a framework for RTL synthesis tools. It has extensive Verilog-2005 support with a basic set of synthesis algorithms for many application domains. This framework can perform any synthesis job by combining existing algorithms using sythensis scripts and additional algorithms as needed. This can be done by extending yosys's C++ code base.

YosysHQ's nextpnr is a versatile open-source framework for timing-driven place-and-route targeting real-world FPGA silicon on Linux, Windows, and macOS [19]. Unlike tools using flat file formats, nextpnr employs an Application Program Interface (API) for architecture implementation, ensuring flexibility for custom functionalities tailored to each FPGA's intricacies. Its JSON-based design interchange format facilitates netlist processing, supporting features like BLIF. Architecture-specific stages handle packing, placement, and routing, with support for timing-driven algorithms. The iCE40 and ECP5 flows utilize data from open-source projects for independence from vendor tools. It also offers a modern GUI for easy navigation and features like database deduplication and a Python API for custom design flows and plugin development.

III. VIDEO GAME EMULATION BASED ON C

A. Development methodology

There are two main parts of the solution: hardware and software. The first development track was the construction of Basys 3 SOC with soft-core CPU, complemented with special, simple, yet capable Mega-Character (MC) Video Controller for game visualization. And the second development track was working out the game algorithm in the comfortable WinOS PC Visual Studio setting, where the hi-res graphic output was emulated on a low-res terminal.

B. Block diagram

Figure 2 shows a block diagram of the entire system. The basic elements of the system are:

- PicoSoC: A RISC-V-based microprocessor core used as the system CPU. It unifies the entire system.
- VGA controller: Control the generation of video signals using the VGA standard. It controls screen resolution, color depth, and refresh rate.
- UART Module: Enables asynchronous serial communication between the FPGA and external devices such as computers.
- Buttons module: Provides user interface via physical pushbuttons on Basys3 (directly influence the movement of the tetrominos on the screen).

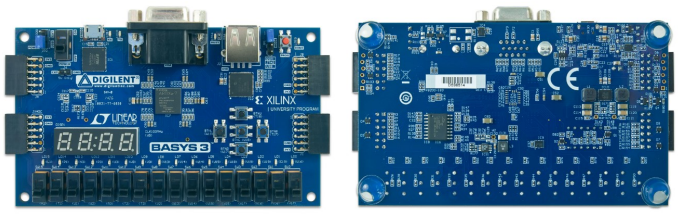


Fig. 1: Basys 3 FPGA development board.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:59 UTC from IEEE Xplore. Restrictions apply.

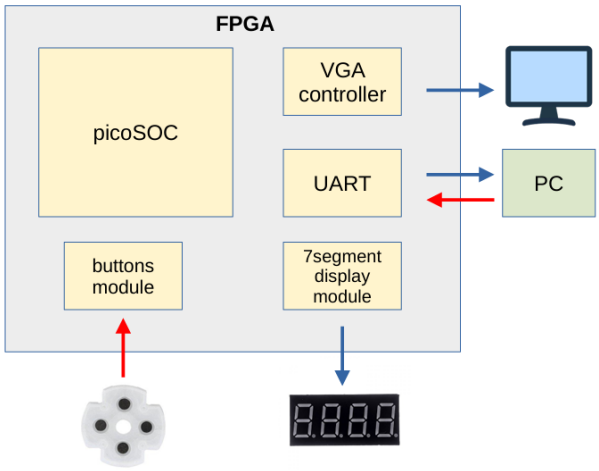


Fig. 2: Block diagram of system.

- 7-segment display module: Control seven-segment LED displays, which were used to display some numerical values.

C. Software - game

TetriSaraj is a Tetris-like game [20] implemented for this work in which the pieces (tetrominoes) are entering the field from the sides, in the random fashion, leaving less time to think about approach and strategy. While the pieces are "falling" horizontally from both sides, one at the time, the logic is otherwise the same as ordinary Tetris. The objective is to complete vertical lines by filling them with tetrominoes.

The SW parts are: Game logic, Game controls, Timing and Rendering. Game logic is not hardware dependent, but other parts are, and they have to be adapted to it through porting process from PC host to embedded CPU.

1) Game logic: It is about the algorithm basics, i.e. the logic of the game. It's worth emphasizing that this the main part of the app and stays the same for both full-fledged PC and bare-metal RISC-V implementation.

Tetrominoes are the essence of the Game Logic element. They're illustrated in figure 3.

There are 7 primary tetromino shapes. Including rotations, there are 28 different shapes. However, to save memory, only the original shapes are stored, using a 7x16 matrix of 1s and 0s, where 1s represent blocks, and 0s represent empty space. For example, the 2nd and the 3rd tetromino, would be coded like:

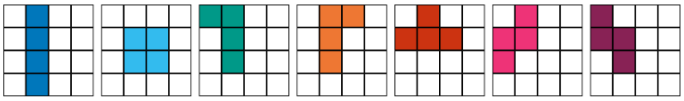


Fig. 3: Tetrominoes - game objects.

$$T2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad T3 = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

These clearly represent arrays of 16 elements. Seven such arrays make the full primary tetromino set. The entire 40x30 playing field is stored in a similar array, for which "field" variable was used.

So, how can this 7x16 primary tetromino matrix account for all 28 rotations? The answer is: using algorithmic smarts in lieu of the brute force. For that, rotate function was used with 3 input arguments: X coordinate, Y coordinate, and Rotation. This function returns the index of the so-rotated block in the original piece. It should be noted that the X, Y coordinates take values from 0..3; rotation (R) can be 0, 90, 180, 270; and returned index is 0..15. Rotation is executed clockwise and is shown on figure 4.

The 1st piece shown on figure 4 is the original, primary tetromino. The remaining 3 are its rotations. To find the Index in the original piece for the corresponding mutant with coordinates X=2, Y=1, R=90° (index 6 in rotated piece), the relation between shapes rotated by 90 degrees should be defined as:

$$Index = 12 + Y - (X * 4) \tag{1}$$

Which means that this specific block is on Index=5 of the original piece.

The same applies to other rotations and other figures. In case of rotation R=270°, block on coordinates X=1, Y=2, where the relation is defined as:

$$Index = 3 - Y + (X * 4) \tag{2}$$

the index will be 5 again, meaning that this specific block is the same as the block on index 5 of the original piece. As for other cases of rotations, R=0° and R=180°, relations are respectively defined as:

$$Index = Y * 4 + X \tag{3}$$

$$Index = 15 - (Y * 4) - X \tag{4}$$

So the complete game logic can be summed up in couple of steps. Upon receiving a game tick, first look is at the commands. If one of the command buttons is pressed, the piece will try to move or rotate in the specified direction. For that, the check is needed: whether the moved-or-rotated piece can fit in the position the command wants it to be in. If it cannot

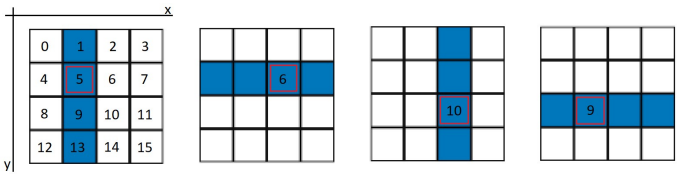


Fig. 4: Block rotation.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:59 UTC from IEEE Xplore. Restrictions apply.

fit, the piece will be locked in the field (write a shape-specific values in the field variable), i.e. don't honor the command.

If it can fit, the command is honored, and check whether any vertical lines were formed is executed. If yes, X coordinates of the formed lines are stored so they can be deleted. The erasing of the vertical is straightforward – the current line is set to empty and all lines "above" it are moved to one place "below". When all this is done, a new random piece is generated.

2) Game controls: On the PC side "GetAsyncKeyState" method was used to detect if a keyboard button was pressed. Upon detecting that a key was pressed, the first check is to see whether the piece can be moved at all. That's covered by "DoesPieceFit" function (line 611 on github: https://github.com/chili-chips-ba/openXC7-TetriSaraj/blob/main/2.sw/main.c).

The same logic applies to Basys3, where FPGA provides a GPIO register for the pushbuttons, from which we then read the button values. This is accomplished by "GetButtonsState" function (line 82 on aforementioned github).

3) Timing: The timing of the game is controlled by it's own ticks. As the game progresses, the game ticks become more frequent, making it more challenging for the player.

On the PC side (implemented in C++), "this_thread::sleep_for" method was used, where sleep is 50 ms, and also a counter which is incremented after sleep. When the counter gets to a certain value (variable named "speed") the piece moves. This is a game tick. As the game progresses and some blocks are stacked, the speed value decreases, which is perceived as a faster play.

A similar method was used for Basys 3. Except that in the absence of "this_thread::sleep_for" there, specific embedded sleep methods were implemented. Other than that, the rest of the logic is the same.

4) Rendering: For video on the PC side, the Windows.h library was used to create a console screen and draw on it. This drawing is based on the "field" variable, which keeps the complete info about the playing field, with all the shapes on it, including the current game state.

The video rendering on the FPGA side (explained in the next chapters) is a bit more complicated and involves writing into MC Frame Buffer registers from the same "field" variable.

IV. HW/SW INTERFACE

This HW/SW setup is more than just a TetriSaraj game. It is a plaform, the solid foundation for developing other apps and retro games in the 40x30 Mega-Character (MC) Frame Buffer paradigm, yet with the full resolution graphics of the host board. We support 16 unique MCs. The MCs can be viewed as mini-Sprites with course XY positioning.

Given Basys3 max resolution of 640x480 RGB444, one MC in this case is a 16x16 block with 4096 colors for each of its 256 dots. That is a rather impressive level of detail, given the meager memory footprint of our MC Frame Buffer. It would otherwise not be possible to accommodate such a resolution even in the comparatively resourceful Artix7-35T.

For example, the 4x smaller and 12x less expensive Tang-Nano9K could also easily host our game. On it, and without having to change the game logic, the same 40x30 MC Frame Buffer would be rendered on the considerably larger 1280x720 RGB888 HDMI screen, by using 32x24 MCs with 16,777,216 colors for each of its 768 dots. Other than the static load of the MCs, and slightly modified RTL for the video back-end, the rest remains the same. This makes for low-effort porting.

A. Screen Organization

The video interface between hardware and software consists of:

- Screen of 640x480 RGB444 pixels divided into 40x30 (=1200) squares, each square being 16x16 pixels.
- Hardware implements a character set of 16 elements. Each element is a 16x16 pixel image in full RGB444 color palette, requiring 384 bytes of memory for one element.
- Software can program each of these 16 unique characters with arbitrary content.
- The software uses these 16 arbitrary characters through a 40x30 FrameBuffer, where each location is represented by 4 bits, indicating to the hardware which of the 16 characters to display at that location.

The image illustrating the screen organization is shown in figure 5.

B. Memory map

One of the first system design steps is to create a Memory Map that governs the interaction between the CPU software and the hardware responsible for rendering the 40x30 = 1200 elements on the screen. This Memory Map can be viewed as

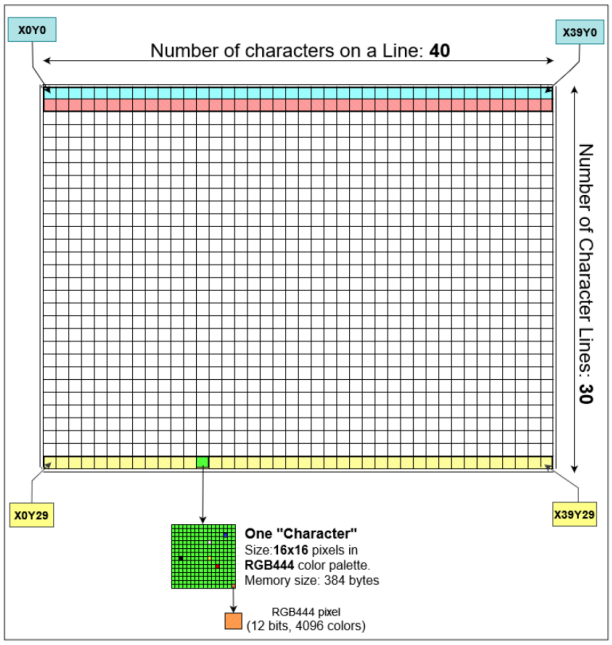


Fig. 5: Screen organization.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:59 UTC from IEEE Xplore. Restrictions apply.

Base Address + Offset	Bit[31:4] – not used	Bit[3:0]	Line
0x0000		x0y0	First line (y=0)
0x0004		x1y0	
0x0008		x2y0	
...		...	
0x004C		x19y0	
0x0050		x20y0	
...		...	
0x009C		x39y0	
0x00A4		x0y1	Second line (y=1)
0x00A8		x1y1	
0x00AC		x2y1	
...		...	
0x0140		x39y0	
0X0144		x0y2	Third line (y=2)
...		...	
0X01E0		x39y2	
...		...	...
0x1220		x0y29	Last line (y=29)
...		...	
0x12BC		x39y29	

Fig. 6: HW/SW interface - Memory Map of the 40x30 Character Frame Buffer.

Base Address + Offset	Bit[31:12] – not used	Bit[11:0]	Mega-character	
0x0000		MCHAR#0.x0y0		
0x0004		MCHAR#0.x1y0		
0x0008		MCHAR#0.x2y0		
0x000C		MCHAR#0.x3y0		
0x0010		MCHAR#0.x4y0		
0x0014		MCHAR#0.x5y0		
0x0018		MCHAR#0.x6y0		
0x001C		MCHAR#0.x7y0		
0x0020		MCHAR#0.x8y0	0	First line (y=0)
0x0024		MCHAR#0.x9y0		
0x0028		MCHAR#0.x10y0		
0x002C		MCHAR#0.x11y0		
0x0030		MCHAR#0.x12y0		
0x0034		MCHAR#0.x13y0		
0x0038		MCHAR#0.x14y0		
0x003C		MCHAR#0.x15y0		
0x0040		MCHAR#0.x0y1		
...		...	0	Second line (y=1)
0x007C		MCHAR#0.x15y1		
0x03C0		MCHAR#0.x0y15		
...		...	0	Last line (y=15)
0x03FC		MCHAR#0.x15y15		
0x0400		MCHAR#1.x0y0		
0x0404		MCHAR#1.x1y0	1	First line (y=0)
...				
...		...		
...		...	15	Last line (y=15)
0x3FFC		MCHAR#F.x15y15		

Fig. 7: Memory Map - Character Set.

the Instruction Set Architecture (ISA) of our graphic mini-controller, serving as a contract or interface between the software and hardware components. The Memory Map for the character frame buffer (CFB) stores information about the screen display. As mentioned above, there are 1200 elements on the screen filled with user-created characters. The Character Set consists of 16 unique, fully programmable characters. In this context, each element on the screen requires 4 bits of memory. The 4-bit code in the CFB is the ID of the character to show on the corresponding XY location.

By organizing the memory in this manner and establishing the Memory Map, we enable effective communication between CPU software and hardware, facilitating the rendering of graphics on the screen. This approach allows the CPU to access and manipulate the graphical data in a structured and predictable manner, enabling smooth and controlled display updates. The memory Map of the 40x30 Character Frame Buffer is shown in figure 6.

When creating characters, the goal is to be able to color each individual pixel in any color. In this regard, a memory map for the character set has been created, implemented in a way that utilizes only the lower 12 bits of a 32-bit word.

The main reason for using such memory maps for the character set and CFB is the simplification of indexing during memory writing. The memory map for the character set is shown in figure 7.

V. Emulation of video game based on FPGA

Table I offers a comprehensive overview of the design's composition and resource utilization. It includes metrics such as the count of wires, memory units, logic cells, and their types, shedding light on the design's complexity and the distribution of its components. Additionally, Table I illustrates the distribution of logic cells, including various types of Look-Up Tables (LUTs), Random Access Memory (RAM) blocks, reflecting the design's logic and memory resource allocation and the estimated number of logic cells required for the design.

In this section, the final results of the work are presented. The objective of emulating a video game using FPGAs has been successfully achieved. The results are shown in figure 8. The complete system contains Basys 3 board (main part of the system - logic and commands), PC (programming and supplying the board) and a monitor (displaying the game).

VI. Conclusion

The outcome of the work was successful when the desired objectives were achieved. There were three goals:

- Game emulation with limited resources,
- Use of open-source tools,
- Establishment of an adaptive platform.

TABLE I: Summary of Design Statistics.

Parameter	Value
Number of wires	5183
Number of wire bits	17990
Number of public wires	485
Number of public wire bits	4379
Number of cells	6427
LUT2	826
LUT3	546
LUT4	518
LUT5	1212
LUT6	549
RAM32M	12
RAM64M	256
RAMB18E1	1
RAMB36E1	16
Estimated number of LCs	2825

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:59 UTC from IEEE Xplore. Restrictions apply.

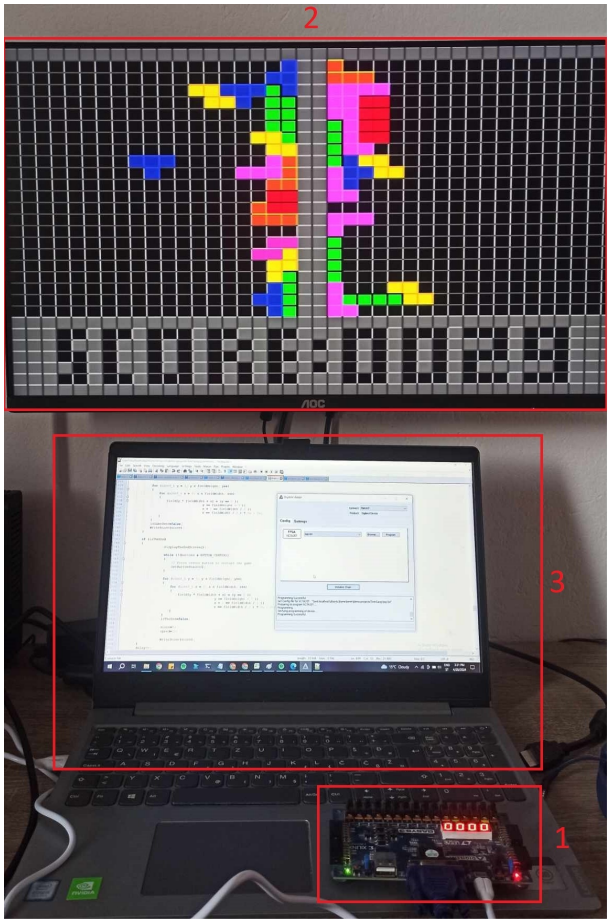


Fig. 8: Game system - Basys 3 (1), monitor (2), PC (3).

So the work demonstrated the feasibility of video game emulation using Basys 3 development board, despite it's small and limited resources. The successful implementation of the game highlighted the capabilities and efficiency of the board. Besides providing valuable insights into hardware-software co-design, this work showcased the potential of FPGA based systems in game development realm. Future work could explore the potential for further refinement and optimization, thus expanding the scope of what can be achieved with Basys 3 development board and FPGA-based systems in general.

Additionally, a key objective of the work was the use of open-source tools, which was achieved with *openXC7 toolchain*. In addition to promoting transparency and collaboration, this approach demonstrates the flexibility and power of open-source software in development. The successful completion of the work using these tools highlighted their capabilities and also set a precedent for their increased use in future work.

The primary objective of this work was the development of an adaptive platform. The successful implementation of the game on the Basys 3 board has resulted in the creation of a versatile and adaptive system. This achievement not only facilitates game development, but also expands the potential for a diverse range of applications beyond this context. This work provided a robust foundation for future improvements and modifications, demonstrating the versatility of this platform in various fields.

REFERENCES

[1] R. Joost and R. Salomon, "Advantages of fpga-based multiprocessor systems in industrial applications," in *31st Annual Conference of IEEE Industrial Electronics Society, 2005. IECON 2005.*, pp. 6–pp, IEEE, 2005.

[2] S. M. S. Trimberger, "Three ages of fpgas: a retrospective on the first thirty years of fpga technology: this paper reflects on how moore's law has driven the design of fpgas through three epochs: the age of invention, the age of expansion, and the age of accumulation," *IEEE Solid-State Circuits Magazine*, vol. 10, no. 2, pp. 16–29, 2018.

[3] D. F. Bacon, R. Rabbah, and S. Shukla, "Fpga programming for the masses," *Communications of the ACM*, vol. 56, no. 4, pp. 56–63, 2013.

[4] R. Cieszewski, M. Linczuk, K. Pozniak, and R. Romaniuk, "Review of parallel computing methods and tools for fpga technology," in *Photonics Applications in Astronomy, Communications, Industry, and High-Energy Physics Experiments 2013*, vol. 8903, pp. 596–608, SPIE, 2013.

[5] A.-L. Georgescu, A. Pappalardo, H. Cucu, and M. Blott, "Performance vs. hardware requirements in state-of-the-art automatic speech recognition," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2021, no. 1, p. 28, 2021.

[6] V.-L. Dao, V.-D. Nguyen, H.-D. Nguyen, and V.-P. Hoang, "Hardware implementation of mfcc feature extraction for speech recognition on fpga," in *International Conference on Advances in Information and Communication Technology*, pp. 248–254, Springer, 2016.

[7] G. Wassi, S. Iloga, O. Romain, and B. Granado, "Fpga-based real-time mfcc extraction for automatic audio indexing on fm broadcast data," in *2015 Conference on Design and Architectures for Signal and Image Processing (DASIP)*, pp. 1–6, IEEE, 2015.

[8] D. G. Bailey, *Design for embedded image processing on FPGAs*. John Wiley & Sons, 2023.

[9] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, J. Zambreno, and P. H. Jones, "Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels," in *2019 IEEE international conference on embedded software and systems (ICESS)*, pp. 1–8, IEEE, 2019.

[10] H. S. Neoh and A. Hazanchuk, "Adaptive edge detection for real-time video processing using fpgas," *Global Signal Processing*, vol. 7, no. 3, pp. 2–3, 2004.

[11] C. Kelly, *Programming 2D Games*. Taylor & Francis, 2012.

[12] M. Stetz, "Leveling up emulation," MSC thesis, New York University, 2022.

[13] K. Liu, Y. Yang, and Y. Zhu, "Tetris game design based on the fpga," in *2012 2nd International Conference on Consumer Electronics, Communications and Networks (CECNet)*, pp. 2925–2928, 2012.

[14] L. Yinuo, Z. Yasong, and W. Ying, "Design of a ping-pong game machine based on vhdl language," in *2023 IEEE International Conference on Sensors, Electronics and Computer Engineering (ICSECE)*, pp. 102–105, 2023.

[15] T. Sato and T. Narumi, "Acceleration of othello computer game using an fpga tablet," in *2015 Third International Symposium on Computing and Networking (CANDAR)*, pp. 581–584, 2015.

[16] C. J. Jiménez-Fernández, C. B. Oliva, P. P. Fernández, A. G. Soto, F. E. P. Ordóñez, and M. V. Barrero, "Learning vhdl through teamwork fpga game design," in *2020 XIV Technologies Applied to Electronics Teaching Conference (TAEE)*, pp. 1–5, 2020.

[17] A. Jahanshahi, M. K. Taram, and N. Eskandari, "Blokus duo game on fpga," in *The 17th CSI International Symposium on Computer Architecture Digital Systems (CADS 2013)*, pp. 149–152, 2013.

[18] "Digilent - basys 3 artix-7 fpga trainer board," 2024. Accessed: 2024-04-24.

[19] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanovic, "Yosys+ nextpnr: an open source framework from verilog to bitstream for commercial fpgas," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 1–4, IEEE, 2019.

[20] H. J. Hoogeboom and W. A. Kosters, "The theory of tetris," *Nieuwsbrief van de Nederlandse Vereniging voor Theoretische Informatica*, vol. 9, pp. 14–21, 2005.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:53:59 UTC from IEEE Xplore. Restrictions apply.